

AI-Native Automation Platform Timeline & Technical Implementation

Architecture Decision: Queueing

As requested, **RabbitMQ has been removed** from the architecture. Instead, we will use **In-Memory Caching / Queueing**. For a single-node setup, this will rely on Go channels and in-memory structures. For a distributed setup (to support the "Unlimited Workers" requirement), we will use **Redis** (which is already in your tech stack) using Redis Streams or Pub/Sub as an in-memory message broker.

Phase 1 (V1): Core Foundation (Months 1-3)

Focus: Basic workflows, authentication, execution engine.

- **Authentication & Organizations (Login, OAuth, SSO, Workspaces)**
 - **Tech:** Next.js for UI forms. Go (Fiber) for backend REST APIs handling JWTs. PostgreSQL for storing users, orgs, and roles.
- **Workflow Builder (Canvas, Nodes, Edges, Groups)**
 - **Tech:** Next.js with `React Flow` for the visual drag-and-drop canvas. `Zustand` for client-side state management. `TanStack Query` for API data fetching.
- **Execution Engine (Workflow execution, Retries, Parallel execution)**
 - **Tech:** Go-based engine using goroutines for parallel execution. Instead of RabbitMQ, a lightweight in-memory queue (Go channels) or Redis Streams will distribute tasks to the Worker Pool.
- **Scheduler & Timeouts**
 - **Tech:** `Go context` for execution timeouts and cancellation. `robfig/cron` (or similar Go library) for scheduling periodic workflow executions.
- **Execution Logs**
 - **Tech:** Buffered in memory (Redis/Go) and batch-inserted into PostgreSQL to handle high throughput without blocking executions.
- **Secrets Management**
 - **Tech:** Encrypted at rest in PostgreSQL using AES-GCM encryption within the Go backend before storage.
- **AI Generator**
 - **Tech:** Python (FastAPI + PydanticAI) microservice. Takes user prompts, queries an LLM, and returns a structured JSON workflow definition that the Go backend

saves and renders.

- **Core Connectors (HTTP, Webhook)**

- **Tech:** Webhooks implemented as dynamic Go Fiber endpoints. HTTP nodes use Go's `net/http` client.

Phase 2 (V2): Collaboration & Ecosystem (Months 4-6)

Focus: Templates, versioning, marketplace, expanding integrations.

- **Marketplace & Templates**

- **Tech:** Next.js UI grid displaying templates stored in PostgreSQL. API Gateway routes requests to fetch template JSONs.

- **Git & Versioning**

- **Tech:** Store workflow JSON snapshots in PostgreSQL with incremental version numbers. Go handles diffing logic.

- **Live Collaboration**

- **Tech:** WebSockets in Go/Fiber syncing state with a frontend CRDT (Conflict-free Replicated Data Type) library like Yjs alongside React Flow.

- **More Connectors (Database, AI, Messaging)**

- **Tech:** Modular internal Go packages that map to external APIs (e.g., Slack, Gmail). Database connector uses Go SQL drivers.

Phase 3 (V3): Enterprise & Advanced AI (Months 7-9)

Focus: Enterprise features, autonomous agents, billing.

- **Agents (Agent Builder & Prompt Templates)**

- **Tech:** Python AI microservice orchestrates multi-step reasoning loops. It sends discrete actions (tool calls) back to the Go execution engine.

- **Enterprise (LDAP, RBAC, Audit Logs, Approval Flow)**

- **Tech:** Go backend integrating with SAML/OIDC. Casbin or custom middleware for RBAC. Audit logs sent asynchronously to an external system (e.g., Loki/Tempo).

- **SDK & Plugin Store**

- **Tech:** Expose platform capabilities via gRPC endpoints. Allow developers to write custom nodes using a TypeScript/Go SDK.

- **Analytics & Billing**

- **Tech:** Next.js charts (Tremor/Tailwind) querying aggregated execution data. Go backend integrates with Stripe/Paddle for marketplace and SaaS billing.